Jupyter Notebook Execution Report

Name: Your Name

Date: June 03, 2026

Cell 1: ■ Markdown

1. Setup and Initialization

In this section, we import the necessary Python libraries for data manipulation (Pandas, NumPy) and data visualization (Matplotlib, Seaborn). We also set the default Seaborn theme to make our charts look better.

Cell 2: ■ Code

```
# Import the necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.set_theme(style='whitegrid')
```

Cell 3: ■ Markdown

2. Data Loading

Here, we load the raw dataset containing the board game information from a CSV file into a Pandas DataFrame so that we can manipulate and analyze it.

Cell 4: ■ Code

```
# 1. Load the base dataset
# Adjust the path if your notebook is in a different directory than your data
file_path = './raw_data/games.csv'
games_df = pd.read_csv(file_path)
```

Cell 5: ■ Markdown

3. Initial Data Inspection

We preview the first 5 rows of our dataset to understand its structure and verify it has loaded correctly. It shows various numerical and categorical features of the games.

Cell 6: ■ Code

```
# Display the first few rows to verify it loaded correctly
display(games_df.head())
```

Output:

oduce | date | large box cardboard edition chipboard board tile special power variant tile inspire german edition | avalon hill sell right game acquire hasbro | date | large b

Cell 7: ■ Markdown

4. Data Cleaning and Exploration

In this step, we clean the dataset to handle missing values and remove any duplicate entries. We use placeholder strings for missing textual data and the median for numerical variables like recommended age or language ease. Then, we print out some basic summary statistics.

Cell 8: ■ Code

```
# 2. Data Cleaning & Exploration
print("--- Data Cleaning & Exploration ---")
print(f"Initial shape: {games_df.shape}")
```

```
print("\nMissing values per column:")
missing_values = games_df.isnull().sum()
print(missing_values[missing_values &gt; 0])
```

```
# Basic Data Cleaning
```

```
games_df = games_df.dropna(how='all')
games_df = games_df.drop_duplicates(subset=['BGGId'])
```

```
# Handle missing values intelligently
games_df['Description'] = games_df['Description'].fillna('No description
available.')
games_df['Family'] = games_df['Family'].fillna('None')
games_df['ImagePath'] = games_df['ImagePath'].fillna('')
games_df['ComAgeRec'] =
games_df['ComAgeRec'].fillna(games_df['ComAgeRec'].median())
games_df['LanguageEase'] =
games_df['LanguageEase'].fillna(games_df['LanguageEase'].median())
```

```
print(f"\nShape after basic cleaning: {games_df.shape}")
```

```
print("\nData summary statistics:")
display(games_df.describe(include='all'))
```

```
print("\nCSV columns and data types:")
for col in games_df.columns:
print(f"- {col}: {games_df[col].dtype}")
# Show values for categorical or small discrete numeric features
if games_df[col].nunique() &lt; 15:
print(f" Unique Values: {games_df[col].unique().tolist()}")
```

Output:

```
--- Data Cleaning & Exploration ---
Initial shape: (21925, 48)
Missing values per column:
Description         1
ComAgeRec        5530
LanguageEase     5891
Family          15262
ImagePath          17
dtype: int64
Shape after basic cleaning: (21925, 48)
Data summary statistics:
```

MfgPlaytime	ComMinPlaytime	ComMaxPlaytime	MfgAgeRec	NumUserRatings	NumComments	NumAlternates
21925.0	21925.0	21925.0	21925.0	21925.0	21925.0	21925.0
nan	nan	nan	nan	nan	nan	nan
nan	nan	nan	nan	nan	nan	nan
nan	nan	nan	nan	nan	nan	nan
90.51352337514253	63.67858608893957	90.51352337514253	9.613409350057012	861.6683238312429	0.0	1.60378563283922
529.6573894102868	443.91621219336093	529.6573894102868	3.6415599211751775	3638.6808570545336	0.0	9.61936382792299
0.0	0.0	0.0	0.0	30.0	0.0	0.0
25.0	20.0	25.0	8.0	56.0	0.0	0.0
45.0	30.0	45.0	10.0	123.0	0.0	0.0
90.0	60.0	90.0	12.0	395.0	0.0	1.0
60000.0	60000.0	60000.0	25.0	108101.0	0.0	850.0

```
CSV columns and data types:
- BGGId: int64
- Name: str
- Description: str
- YearPublished: int64
- GameWeight: float64
- AvgRating: float64
- BayesAvgRating: float64
- StdDev: float64
- MinPlayers: int64
  Unique Values: [3, 2, 1, 4, 8, 0, 6, 5, 10, 7, 9]
- MaxPlayers: int64
- ComAgeRec: float64
- LanguageEase: float64
- BestPlayers: int64
  Unique Values: [5, 0, 3, 4, 6, 2, 7, 8, 1, 13, 12, 15, 14, 9]
- GoodPlayers: str
- NumOwned: int64
- NumWant: int64
- NumWish: int64
- NumWeightVotes: int64
```

- MfgPlaytime: int64
- ComMinPlaytime: int64
- ComMaxPlaytime: int64
- MfgAgeRec: int64
- NumUserRatings: int64
- NumComments: int64
  Unique Values: [0]
- NumAlternates: int64
- NumExpansions: int64
- NumImplementations: int64
- IsReimplementation: int64
  Unique Values: [0, 1]
- Family: str
- Kickstarted: int64
  Unique Values: [0, 1]
- ImagePath: str
- Rank:boardgame: int64
- Rank:strategygames: int64
- Rank:abstracts: int64
- Rank:familygames: int64
- Rank:thematic: int64
- Rank:cgs: int64
- Rank:wargames: int64
- Rank:partygames: int64
- Rank:childrensgames: int64
- Cat:Thematic: int64
  Unique Values: [0, 1]
- Cat:Strategy: int64
  Unique Values: [1, 0]
- Cat:War: int64
  Unique Values: [0, 1]
- Cat:Family: int64
  Unique Values: [0, 1]
- Cat:CGS: int64

```
  Unique Values: [0, 1]
- Cat:Abstract: int64
  Unique Values: [0, 1]
- Cat:Party: int64
  Unique Values: [0, 1]
- Cat:Childrens: int64
  Unique Values: [0, 1]
```

Cell 9: ■ Markdown

5. Feature Engineering

We create new features that will be critical for our recommendation model:

- `TotalEngagement`: Combining ownership, wishes, wants, and user ratings as a proxy for the board game's popularity.

- `MaxPlaytime`: Consolidating missing values in maximum playtimes to have a reliable metric.

Cell 10: ■ Code

```
# 3. Feature Engineering: Engagement & Playtime
print("\n--- Feature Engineering ---")
games_df['TotalEngagement'] = games_df['NumOwned'] + games_df['NumWant'] +
games_df['NumWish'] + games_df['NumUserRatings']
```

```
# Consolidate playtime features (using ComMaxPlaytime as an upper bound proxy,
fallback to MfgPlaytime)
games_df['MaxPlaytime'] = games_df['ComMaxPlaytime'].replace(0,
np.nan).fillna(games_df['MfgPlaytime'])
```

```
display(games_df[['Name', 'TotalEngagement',
'MaxPlaytime']].sort_values(by='TotalEngagement', ascending=False).head())
```

Output:

```
--- Feature Engineering ---
```

Index	Name	TotalEngagement	MaxPlaytime
7919	Pandemic	284552	45.0
12	Catan	279141	120.0

Index	Name	TotalEngagement	MaxPlaytime
673	Carcassonne	275530	45.0
9861	7 Wonders	221519	30.0
14811	Codenames	197544	15.0

Cell 11: ■ Markdown

6. Category Analysis

Since our board game suggester heavily relies on categories (the `Cat:*` features), we extract these category columns and plot a bar chart. This visualizes the overall representation of games across categories such as Strategy, Thematic, or Party.

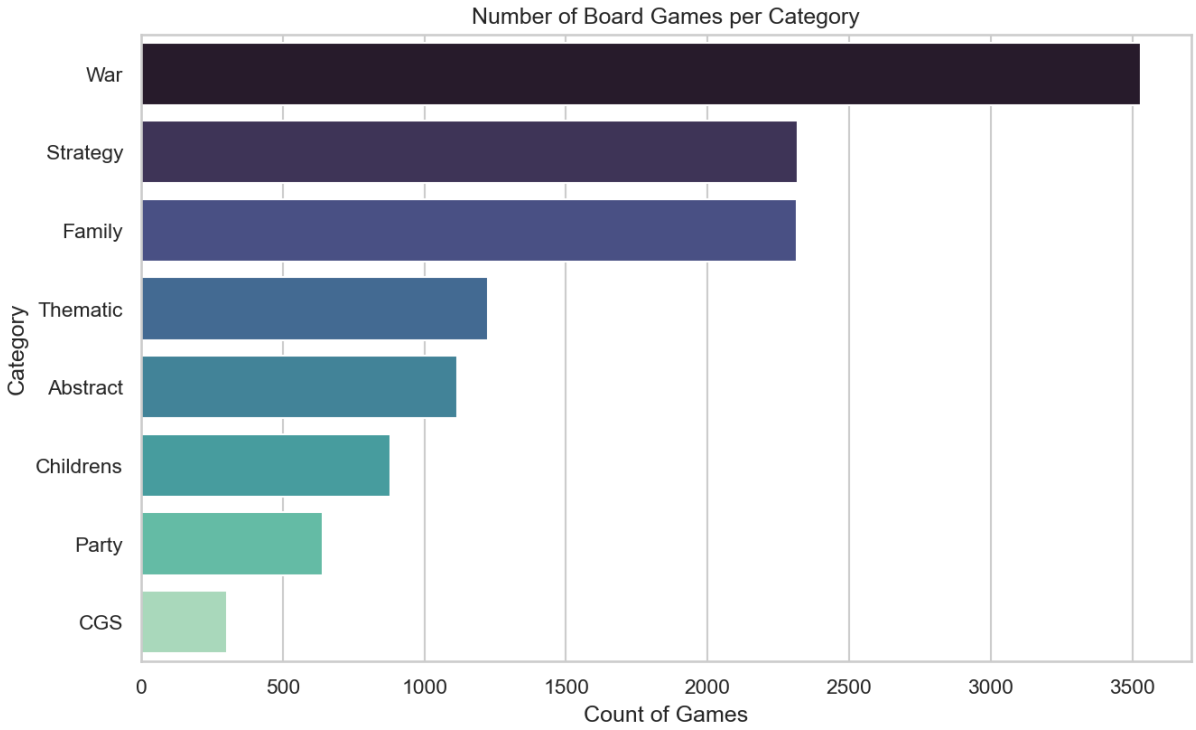
Cell 12: ■ Code

```
# 4. Analyze Category representations for our Suggester Model
cat_columns = [col for col in games_df.columns if col.startswith('Cat:')]
cat_counts = games_df[cat_columns].sum().sort_values(ascending=False)
```

```
plt.figure(figsize=(10, 6))
sns.barplot(x=cat_counts.values, y=[c.replace('Cat:', '') for c in
cat_counts.index], palette='mako')
plt.title('Number of Board Games per Category')
plt.xlabel('Count of Games')
plt.ylabel('Category')
plt.show()
```

Output:

```
[STDERR]
&lt;string&gt;:6: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign th
&lt;string&gt;:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
```



Cell 13: ■ Markdown

7. Correlation Analysis

Finally, we plot a heatmap representing the correlation matrix for the key numerical features. This analysis allows us to understand the linear relationships in the data, e.g., if highly complex games (`GameWeight`) tend to have higher ratings (`AvgRating`).

Cell 14: ■ Code

```
# 5. Evaluate correlations between key numerical features
numeric_cols = ['YearPublished', 'GameWeight', 'AvgRating', 'BayesAvgRating',
'MinPlayers', 'MaxPlayers', 'MaxPlaytime', 'ComAgeRec',
'LanguageEase', 'TotalEngagement']
```

```
plt.figure(figsize=(12, 8))
corr = games_df[numeric_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Matrix of Key Numerical Features')
plt.show()
```

Output:

```
[STDERR]
&lt;string&gt;:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
```